Classes and Objects

Name: Rahul Vedant Pemsing
Student ID: 36305529
Date: 30/09/2025

Learning Journey and Evidence

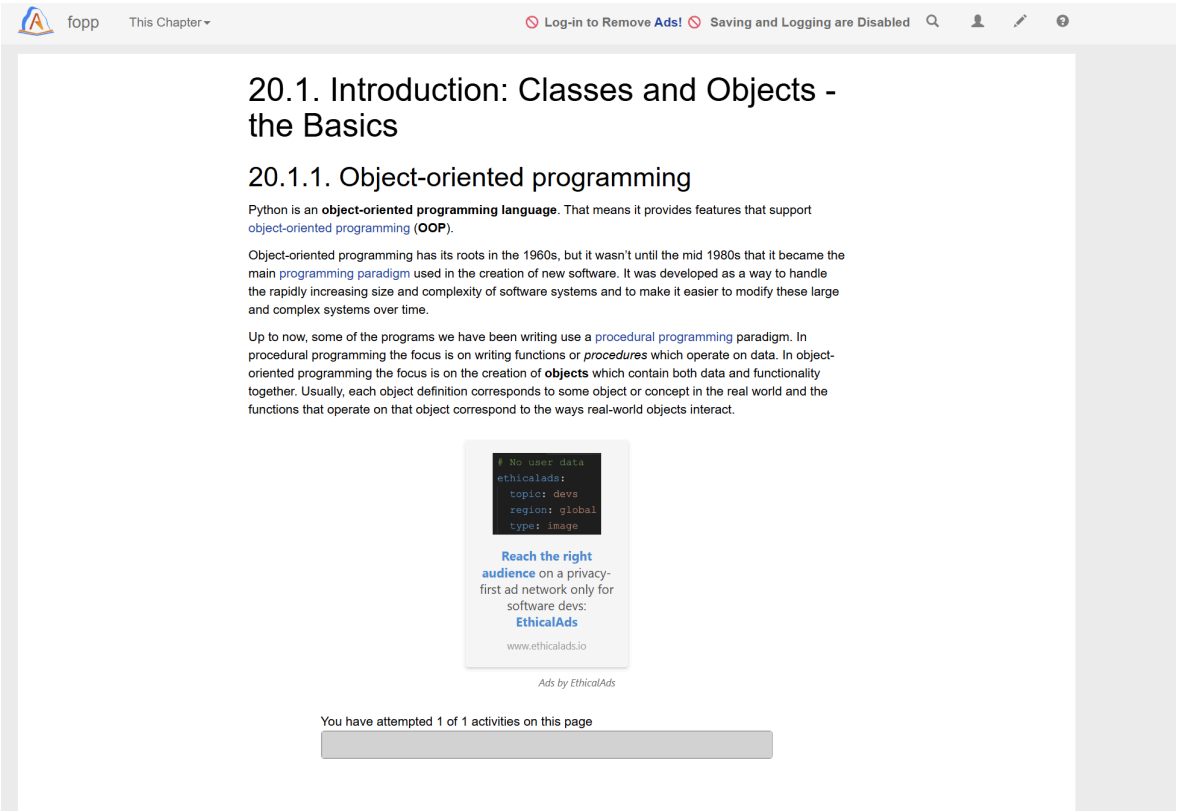
Prior Knowledge and Experience

I am familiar with OOP from the study requirements for my A-levels. I have coded multiple programs that needed to use object oriented programming.

In task C2: Since this requires going over member functions to gain a complete understanding, I had completed much of the work in C2 in the C++ programming language, so for this one(P10) I just had to convert my C++ programs to python.

Study to Acquire, Refresh, and/or Extend Capability

To study and master this topic I did the following:

30/09/2025- Read the python guide and understood how to code OOP in python

I learned the basic syntax for defining classes and objects in Python. Unlike C++, Python does not require explicit declarations for public or private sections, but naming conventions (like using an underscore) are followed to indicate private

fopp This Chapter Log-in to Remove Ads! Saving and Logging are Disabled

20.1. Introduction: Classes and Objects - the Basics

20.1.1. Object-oriented programming

Python is an **object-oriented programming language**. That means it provides features that support object-oriented programming (**OOP**).

Object-oriented programming has its roots in the 1960s, but it wasn't until the mid 1980s that it became the main programming paradigm used in the creation of new software. It was developed as a way to handle the rapidly increasing size and complexity of software systems and to make it easier to modify these large and complex systems over time.

Up to now, some of the programs we have been writing use a procedural programming paradigm. In procedural programming the focus is on writing functions or *procedures* which operate on data. In object-oriented programming the focus is on the creation of **objects** which contain both data and functionality together. Usually, each object definition corresponds to some object or concept in the real world and the functions that operate on that object correspond to the ways real-world objects interact.


```
# No user data
ethicalads:
  topic: devs
  region: global
  type: image
```

Reach the right audience on a privacy-first ad network only for software devs: EthicalAds

www.ethicalads.io

Ads by EthicalAds

You have attempted 1 of 1 activities on this page

fields. I also understood how Python uses the __init__ method as its constructor and how self is used to access instance attributes and methods.

30/09/2025 - Coding[Field Guide → Part 2 → 7.Member function → Guided tour → Account Class & Objects]

Description:

A simple bank account class

```
Class: Bank Account
Fields:
- name: the name of the account. (public)
- balance: the funds within the account. In cents. (private)
Constructors:
 - One that accepts the name and balance, and initialises these.
 - Default constructor sets the name to a default value and the
balance to 0.
Methods:
- deposit: will increase the balance by a given amount.
- withdraw: will decrease the balance by a given amount.
- print: will output its details to the terminal.
- get_balance: will return the current balance.
```

NOTE: The C++ version has already been done in C2. Only the python version will be included here.

My code:

Rahul Pemsing
36305529

acc.py > ...

```python
class Account:
    def __init__(self, name="Unknown", balance=0):
        self.name = name
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if amount > 0 and self.__balance >= amount:
            self.__balance -= amount

    def print(self):
        print(f"{self.name}: {self.__balance} cents")

    def get_balance(self):
        return self.__balance

if __name__ == "__main__":
    acc1 = Account("Alice", 5000)
    acc2 = Account("Bob", 3000)
    acc3 = Account()

    acc1.print()
    acc2.print()
    acc3.print()

    acc1.deposit(1000)
    acc1.withdraw(500)
    acc1.print()

    print(f"Alice's balance: {acc1.get_balance()} cents")
```

Output:

```
Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/myco
$ C:/msys64/mingw64/bin/python3.12.exe c:/Users/Rahu
Alice: 5000 cents
Bob: 3000 cents
Unknown: 0 cents
Alice: 5500 cents
Alice's balance: 5500 cents
```

I learned how to create a class with fields, constructors, and methods in Python. The activity reinforced how to use encapsulation by making the balance private and controlling access to it through methods like deposit, withdraw, and get_balance. It also showed me the practical use of methods for input/output operations, demonstrating how classes can represent real-world entities like a bank account.

30/09/2025 -Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself → User account]

Description:

Create a class to represent a user account. This should have a username and a password. The password should be kept private, so that it cannot be read outside the class. The system will have a single user, which is set when the program starts. Allow the user to test the login, and to change their password. The change password method should accept the current password, and the new password, and only update the password if the existing passwords match. The code should also check if the new password was not the previous password. You may choose to add additional constraints on the password as well.

My code(python: C++ counterpart already done in C2):

user_acc.py > ...

```python
def read_string(prompt):
    """Prompt the user with a message and return their string input."""
    return input(prompt)

def read_integer_limit(prompt, min_val, max_val):
    """Prompt the user to enter an integer within a specified range."""
    while True:
        try:
            value = int(input(prompt))
            if min_val <= value <= max_val:
                return value
            print("Please enter a integer number in the correct range.")
        except ValueError:
            print("Please enter a integer number in the correct range.")

class AccDetails:
    """Represents an account with a username and password."""

    def __init__(self, username, password):
        """Construct a new account with the given username and password."""
        self.username = username
        self.password = password
        self.used_passwords = ["", "", ""]
        print("- Account set.")

    def test_login(self):
        """Prompt the user for username and password, and check login validity."""
        in_username = read_string("Username: ")
        in_password = read_string("Password: ")

        if in_username == self.username and in_password == self.password:
            print("- Login successful")
        else:
            print("- Login failed")
```

```python

    def change_password(self):
        """Change the account password after validating credentials."""
        in_username = read_string("Username: ")
        in_password = read_string("Current password: ")
        new_password = read_string("New password: ")

        if in_username != self.username:
            print("- Failed to change password - invalid username")
        elif in_password != self.password:
            print("- Failed to change password - invalid password")
        elif new_password in self.used_passwords:
            print("- Failed to change password - password reuse")
        else:
            self.used_passwords[0] = self.used_passwords[1]
            self.used_passwords[1] = self.used_passwords[2]
            self.used_passwords[2] = self.password
            self.password = new_password
            print("Password changed successfully!")

def menu():
    """Display the account management menu options."""
    print("\n1: Set account")
    print("2: Test login")
    print("3: Change password")
    print("4: Quit")

def main():
    """Main entry point of the program."""
    account = AccDetails(read_string("Username: "), read_string("Password: "))

    while True:
        menu()
        print()
        choice = read_integer_limit("> ", 1, 4)

        if choice == 1:
            account = AccDetails(read_string("Username: "), read_string("Password: "))
        elif choice == 2:
            account.test_login()
        elif choice == 3:
            account.change_password()
        elif choice == 4:
            break

if __name__ == "__main__":
    main()
```

Output:

```
Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/mycode/my_python_code/user_account
$ C:/msys64/mingw64/bin/python3.12.exe c:/Users/Rahul/Desktop/mycode/my_python_code/user_account/user_acc.py
Username: vrp
Password: ved22
- Account set.

1: Set account
2: Test login
3: Change password
4: Quit

> 2
Username: vrp
Password: ved22
- Login successful

1: Set account
2: Test login
3: Change password
4: Quit

> 3
Username: vrp
Current password: ved22
New password: ved2205
Password changed successfully!

1: Set account
2: Test login
3: Change password
4: Quit

> 2
Username: vrp
Password: ved2205
- Login successful

1: Set account
2: Test login
3: Change password
4: Quit

> 4
```

I learned the importance of keeping sensitive data private, such as a password, and controlling how it is updated. This task showed me how to implement authentication checks within a class and enforce constraints like preventing the reuse of the old password. I also understood how object-oriented programming can help enforce security and good design principles by combining data and the logic that manipulates it.

30/09/2025 - Coding[Field Guide → Part 2 → 7. Member function → Activities → Test yourself → Dice]

Description:

Create a class to represent a die (a single dice). This can have a number of sides, which cannot change after it is created. The die will also have a current value, being the value visible on top when the die is rolled. The die can be rolled, which randomises the value shown on top based on the number of sides. The current value can be read, but not changed by other code. Include a method to print the details of the die, including its sides and current value so that it can be shown in the test code.

My code:

```python
import random

def read_integer(prompt):
    """Function to input an integer"""
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Please enter a integer number.")

def read_integer_limit(prompt, min_val, max_val):
    """Function to input an integer within a range"""
    while True:
        value = read_integer(prompt)
        if min_val <= value <= max_val:
            return value
        print("Please enter a integer number in the correct range.")

class Dice:
    def __init__(self, die_face):
        """constructor of the class dice"""
        self.face = die_face
        self.current_value = 0

    def print_die(self):
        """Prints the value obtain after rolling"""
        print(f"{self.face} sides: value is {self.current_value}")

    def roll_die(self):
        """Rolls the dice"""
        self.current_value = random.randint(1, self.face)

def menu():
    print("\n1: Roll die")
    print("2: Get new die")
    print("3: Quit")
```

```python

def main():
    die = Dice(read_integer("Sides: "))

    while True:
        menu()
        choice = read_integer_limit("> ", 1, 3)

        if choice == 1:
            die.roll_die()
            die.print_die()
        elif choice == 2:
            die = Dice(read_integer("Sides: "))
        elif choice == 3:
            break

if __name__ == "__main__":
    main()
```

Output:

```
Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/mycode/my_python_code/Dice
$ C:/msys64/mingw64/bin/python3.12.exe c:/Users/Rahul/Desktop/mycode/my_python_code/Dice/dice.py
Sides: 6

1: Roll die
2: Get new die
3: Quit
> 1
6 sides: value is 5

1: Roll die
2: Get new die
3: Quit
> 2
Sides: 20

1: Roll die
2: Get new die
3: Quit
> 1
20 sides: value is 18

1: Roll die
2: Get new die
3: Quit
> 3

Rahul@VRP_Legion MINGW64 /c/Users/Rahul/Desktop/mycode/my_python_code/Dice
$
```

I learned how to design a class that models a physical object, in this case, a die. I practiced using randomization to simulate rolling the die and ensuring that the number of sides remains immutable after creation. This taught me about the difference between mutable and immutable properties and how methods can encapsulate behaviors that directly reflect real-world actions.

Brief Summary of Concepts

Concept	Key Idea / Concept
Object	An instance of a class that represents a specific entity with its own data and behavior.
Class	A blueprint or template for creating objects, defining their structure and behavior.
Method	A function defined within a class that operates on its data.
Field	A variable defined inside a class that stores data specific to an object.
Property	A controlled way of accessing fields, often used to enforce encapsulation.
Constructor	A special method (__init__ in Python) that initializes an object when it is created.
Reference type	Objects in Python are stored as references, meaning variables hold references to memory locations rather than the actual data.
Visibility: Public, Private	In Python, public members can be accessed directly, while private members are indicated by a leading underscore and are intended to be accessed only within the class.

Reflection

What is the most important thing you learned from this and why?

The most important thing I learned is how Python implements object-oriented programming differently from C++. Understanding the use of self and the __init__ method was crucial because it forms the foundation for writing reusable and organized code. This knowledge is important because it improves how I design and structure programs, making them easier to maintain and extend.

What differences and similarities do you notice between structs (without member functions) and the way we are using classes now?

Structs without member functions only allow you to group related data, while classes allow both data and behaviors to be grouped together. This means classes are much more powerful as they encapsulate logic with data. Both, however, provide a way to represent entities with multiple attributes.

What differences and similarities do you notice between functions / procedures and methods?

Functions and procedures are standalone blocks of code, while methods are tied to a specific class and operate on its data. Both allow code reuse and abstraction, but methods carry the additional context of the object they belong to, making them more closely aligned with object-oriented principles.

Why is object-oriented programming considered a different paradigm? What does this mean, and how does it influence how you approach learning this?

OOP is considered a different paradigm because it focuses on modeling software around objects rather than just sequences of instructions. This changes the way I think about problems, encouraging me to design solutions that mirror real-world entities and interactions. It also means I must shift from procedural thinking to considering data and behavior together.

How does object oriented programming change the way you think about your software?

OOP changes my approach to software by making me think in terms of reusable components. Instead of writing large blocks of procedural code, I now break problems into smaller, self-contained classes that can interact with each other. This makes my software more modular, easier to debug, and scalable.

References

Generative AI acknowledgement

No AI was used for either the code or this document